[Lecture Note] Overview of the Data Mining Process

MGS3701 Data Mining, Spring 2025

Chad (Chungil Chae)

Sun, 9 March 2025

In this chapter, we give an overview of the steps involved in data mining, starting from a clear goal definition and ending with model deployment. The general steps are shown schematically in Figure 2.1. We also discuss issues related to data collection, cleaning, and preprocessing. We introduce the notion of data partitioning, where methods are trained on a set of training data and then their performance is evaluated on a separate set of validation data, as well as explain how this practice helps avoid overfitting. Finally, we illustrate the steps of model building by applying them to data.(Shmueli, Bruce, Yahav, Patel, & Lichtendahl Jr, 2017)

https://www.youtube.com/watch?v=7rs0i-9nOjo

Introduction

- In this chapter, we introduce the variety of methods sometimes referred to as data mining.
- Data mining focuses on what has come to be called predictive analytics, the tasks of
  - *Classification*
  - *Prediction*
  - *Pattern discovery*

Core Idea in Data Mining

Classification

- Classification is the most basic form of data analysis.
  - e.g., application of a loan, credit card transaction
- To examine data where the classification is
  - unknown or
  - will occur in the future
- Similar to classification is develop rules

Prediction

- Prediction is to predict **the value of a numerical variable** (e.g., amount of purchase) rather than a class (e.g.,purchaser or nonpurchaser).
- the value of a continuous variable.

Assiciation Rules and Recommendation Systems

- **What goes with what**
- Association rules, or affinity analysis, is designed to find such **general associations patterns** between items in large databases.
  - grocery stores: product placement, weekly promotional offers, bundling products.
  - hospital database: which symptom is followed by what other symptom to help predict future symptoms for returning patients.
  - Online recommendation systems: collaborative filtering

> **Collaborative filtering**
>
> Collaborative filteringa method that uses individual users' preferences and tastes given their historic purchase, rating, browsing, or any other measurable behavior indicative of preference, as well as other users' history.

> **Association rules vs Collaborative filtering**
>
> In contrast to association rules that generate rules general to an entire population, collaborative filtering generates "what goes with what" at the individual user level. Hence, collaborative filtering is used in many recommendation systems that aim to deliver personalized recommendations to users with a wide range of preferences.

Predictive Analytics

- Classification, prediction, and to some extent, association rules and collaborative filtering constitute the analytical methods employed in predictive analytics.
- The term predictive analytics is sometimes used to also include data pattern identification methods such as clustering.

Data Reduction and Dimension Reduction

- Data mining algorithms is often improved
  - when the number of variables is limited, and
  - when large numbers of records can be grouped into homogeneous groups.

- For example,
  - rather than dealing with thousands of product types, an analyst might wish to group them into a smaller number of groups and build separate models for each group.
  - Or a marketer might want to classify customers into different "personas," and must therefore group customers into homogeneous groups to define the personas.
- This process of consolidating a large number of records (or cases) into a smaller set is termed data reduction. Methods for reducing the number of cases are often called clustering.
- Reducing the number of variables is typically called dimension reduction.
  - Dimension reduction is a common initial step before deploying data mining methods, intended to improve predictive power, manageability, and inter-pretability.

Data Exploration and Visualization

- Exploration is aimed at
  - understanding the global landscape of the data, and
  - detecting unusual values.
- Exploration is used for **data cleaning** and **manipulation** as well as for **visual discovery** and **hypothesis generation**.
- Methods for exploring data include looking at various data aggregations and summaries
- (Both numerically and graphically) looking at each variable separately as well as looking at relationships among variables.
- The purpose is to **discover patterns** and **exceptions**.
- Data Visualization or Visual Analytics - Exploration by creating charts and dashboards
- For numerical variables,
  - we use histograms and boxplots to learn about the distribution of their values,
    - to detect outliers (extreme observations), and to
    - find other information that is relevant to the analysis task.
- For categorical variables,
  - we use bar charts. We can also look at scatter plots of pairs of numerical variables
    - to learn about possible relationships,
    - the type of relationship,
    - to detect outliers.
- Visualization can be greatly enhanced by adding features such as color and interactive navigation.

Supervised and Unsupervised Learning

- A fundamental distinction among data mining techniques is between supervised and unsupervised methods.

Supervised Learning Algorithms

- Supervised learning algorithms are those used in **classification** and **prediction**.
  - We must have data available in which the value of the outcome of interest (e.g., purchase or no purchase) is known, "labeled data"
  - These training data are the data from which the classification or prediction algorithm "learns," or is "trained," about the relationship between predictor variables and the outcome variable.
  - Once the algorithm has learned from the training data, it is then applied to **another sample of labeled data (the validation data)** where the *outcome is known but initially hidden*, to see how well it does in comparison to other models.
  - If many different models are being tried out, it is prudent to save a third sample, which also includes known outcomes (the test data) to use with the model finally selected to predict how well it will do.
  - The model can then be used to classify or predict the outcome of interest in new cases where the outcome is unknown.

> **Linear Regression as Supervised Machine Learning Algorithm**
>
> - Simple linear regression is an example of a supervised learning algorithm (although rarely called that in the introductory statistics course where you probably first encountered it).
>   - The Y variable is the (known) outcome variable and the X variable is a predictor variable.
>   - A regression line is drawn to minimize the sum of squared deviations between the actual Y values and the values predicted by this line.
>   - The regression line can now be used to predict Y values for new values of X for which we do not know the Y value.

Unsupervised Learning Algorithms

- Unsupervised learning algorithms are those used where there is no outcome variable to predict or classify.
- Hence, there is no "learning" from cases where such an outcome variable is known.
- Association rules, dimension reduction methods, and clustering techniques are all unsupervised learning methods.
- Supervised and unsupervised methods are sometimes used in conjunction.

> **Note**
>
> For example, unsupervised clustering methods are used to separate loan applicants into several risk-level groups. Then, supervised algorithms are applied separately to each risk-level group for predicting propensity of loan default.

The Steps in Data Mining

Data Mining Steps

1. **Develop an understanding of the purpose of the data mining project.** How will the stakeholder use the results? Who will be affected by the results? Will the analysis be a one-shot effort or an ongoing procedure?
2. **Obtain the dataset to be used in the analysis.** This often involves sampling from a large database to capture records to be used in an analysis. How well this sample reflects the records of interest affects the ability of the data mining results to generalize to records outside of this sample. It may also involve pulling together data from different databases or sources The databases could be internal (e.g., past purchases made by customers) or external (credit ratings). While data mining deals with very large databases, usually the analysis to be done requires only thousands or tens of thousands of records.
3. **Explore, clean, and preprocess the data.** This step involves verifying that the data are in reasonable condition. How should missing data be handled? Are the values in a reasonable range, given what you would expect for each variable? Are there obvious outliers? The data are reviewed graphically: for example, a matrix of scatterplots showing the relationship of each variable with every other variable. We also need to ensure consistency in the definitions of fields, units of measurement, time periods, and so on. In this step, new variables are also typically created from existing ones. For example, "duration" can be computed from start and end dates.
4. **Reduce the data dimension, if necessary.** Dimension reduction can involve operations such as eliminating unneeded variables, transforming variables (e.g., turning "money spent" into "spent > \$100" vs. "spent ≤ \$100"), and creating new variables (e.g., a variable that records whether at least one of several products was purchased). Make sure that you know what each variable means and whether it is sensible to include it in the model.
5. **Determine the data mining task.** (classification, prediction, clustering, etc.). This involves translating the general question or problem of Step 1 into a more specific data mining question.
6. **Partition the data (for supervised tasks).** If the task is supervised (classification or prediction), randomly partition the dataset into three parts: training, validation, and test datasets.
7. **Choose the data mining techniques to be used.** (regression, neural nets, hierarchical clustering, etc.).
8. **Use algorithms to perform the task.** This is typically an iterative process—trying multiple variants, and often using multiple variants of the same algorithm (choosing different variables or settings within the algorithm). Where appropriate, feedback from the algorithm's performance on validation data is used to refine the settings.
9. **Interpret the results of the algorithms.** This involves making a choice as to the best algorithm to deploy, and where possible, testing the final choice on the test data to get an idea as to how well it will perform. (Recall that each algorithm may also be tested on the validation data for tuning purposes; in

this way, the validation data become a part of the fitting process and are likely to underestimate the error in the deployment of the model that is finally chosen.)

10. **Deploy the model.** This step involves integrating the model into operational systems and running it on real records to produce decisions or actions. For example, the model might be applied to a purchased list of possible customers, and the action might be "include in the mailing if the predicted amount of purchase is > $10." A key step here is "scoring" the new records, or using the chosen model to predict the outcome value ("score") for each new record.

SEMMA

The foregoing steps encompass the steps in SEMMA, a methodology developed by the software company SAS:

- **Sample**: Take a sample from the dataset; partition into training, validation, and test datasets.
- **Explore**: Examine the dataset statistically and graphically.
- **Modify**: Transform the variables and impute missing values.
- **Model**: Fit predictive models (e.g., regression tree, neural network).
- **Assess**: Compare models using a validation dataset.

IBM SPSS Modeler (previously SPSS-Clementine) has a similar methodology, termed CRISP-DM (CRoss-Industry Standard Process for Data Mining). All these frameworks include the same main steps involved in predictive modeling.

Other Models

- **KDD Model**: Knowledge Discovery in Databases (KDD) is a systematic process that seeks to identify valid, novel, potentially useful, and ultimately understandable patterns from large amounts of data. In simpler terms, it's about transforming raw data into valuable knowledge.
- **CRISP-DM**: CRISP-DM stands for Cross-Industry Standard Process for Data Mining. It is a cyclical process that provides a structured approach to planning, organizing, and implementing a data mining project. The process consists of six major phases: Business Understanding, Data Understanding, Data, Preparation, Modeling, Evaluation, Deployment

https://www.youtube.com/watch?v=q_okDS2RtzY

Preliminary Steps

Organization of Datasets

Datasets are nearly always constructed and displayed so that variables are in columns and records are in rows. We will illustrate this with home values in West Roxbury, Boston, in 2014. Fourteen variables are recorded for over 5000 homes. The spreadsheet is organized so that each row represents a home—the first

home's assessed value was $344,200, its tax was $4430, its size was 9965 ft², it was built in 1880, and so on. In supervised learning situations, one of these variables will be the outcome variable, typically listed in the first or last column (in this case it is **TOTAL VALUE**, in the first column).

Predicting Home Values in the West Roxbury Neighborhood

The Internet has revolutionized the real estate industry. Realtors now list houses and their prices on the web, and estimates of house and condominium prices have become widely available, even for units not on the market. At this time of writing, Zillow (<www.zillow.com>) is the most popular online real estate information site in the United States, and in 2014 they purchased their major rival, Trulia. By 2015, Zillow had become the dominant platform for checking house prices and, as such, the dominant online advertising venue for realtors. What used to be a comfortable 6% commission structure for realtors, affording them a handsome surplus (and an oversupply of realtors), was being rapidly eroded by an increasing need to pay for advertising on Zillow. (This, in fact, is the key to Zillow's business model—redirecting the 6% commission away from realtors and to itself.)

Zillow gets much of the data for its "Zestimates" of home values directly from publicly available city housing data, used to estimate property values for tax assessment. A competitor seeking to get into the market would likely take the same approach. So might realtors seeking to develop an alternative to Zillow.

A simple approach would be a naive, model-less method—just use the assessed values as determined by the city. Those values, however, do not necessarily include all properties, and they might not include changes warranted by remodeling, additions, etc. Moreover, the assessment methods used by cities may not be transparent or always reflect true market values.

However, the city property data can be used as a starting point to build a model, to which additional data (such as that collected by large realtors) can be added later. Let's look at how Boston property assessment data, available from the city of Boston, might be used to predict home values. The data in **WestRoxbury.csv** includes information on single-family owner-occupied homes in West Roxbury, a neighborhood in southwest Boston, MA, in 2014. The data include values for various predictor variables and for an outcome—assessed home value (**TOTAL VALUE**). This dataset has 14 variables, and a description of each variable is given in Table 2.1 (the full data dictionary provided by the City of Boston is available at http://goo.gl/QBRlYF; we have modified a few variable names). The dataset includes 5802 homes. A sample of the data is shown in Table 2.2, and the "data dictionary" describing each variable is in Table 2.1. As we saw earlier, below the header row, each row in the data represents a home. For example, the first home was assessed at a total value of $344.2 thousand (**TOTAL VALUE**). Its tax bill was $4330. It has a lot size of 9965 square feet (ft²), was built in the year 1880, has two floors, six rooms, and so on.

Loading and Looking at the Data in R

To load data into R, we will typically want to have the data available as a CSV (comma-separated values) file. If the data are in an XLS (or XLSX) file, we can save that file in Excel as a CSV file by going to **File > Save as > Save as type: CSV (Comma delimited) (*.csv) > Save**.

Note: When dealing with `.csv` files in Excel, beware of two pitfalls: - Opening a `.csv` file in Excel strips off leading zeros, which can corrupt zipcode data. - Saving a `.csv` file in Excel saves only the digits that are displayed; if you need precision to a certain number of decimals, ensure they are displayed before saving.

Once we have R and RStudio installed on our machine and the `WestRoxbury.csv` file saved as a CSV file, we can run the code in Table 2.3 to load the data into R. Data from a CSV file is stored in R as a **data frame** (e.g., `housing.df`). If our CSV file has column headers, these headers get automatically stored as the column names of our data. A data frame is the fundamental object almost all analyses begin with in R. A data frame has rows and columns: the rows are the observations for each case (e.g., house), and the columns are the variables of interest (e.g., `TOTAL VALUE`, `TAX`).

The code in Table 2.3 walks you through some basic steps you will want to perform prior to doing any analysis:

- Finding the size and dimension of your data (number of rows and columns)
- Viewing all the data
- Displaying only selected rows and columns
- Computing summary statistics for variables of interest

Comments in R are preceded by the # symbol.

Sampling from a Database

Typically, we perform data mining on less than the complete database. Data mining algorithms will have varying limitations on what they can handle in terms of the numbers of records and variables, limitations that may be specific to computing power and capacity as well as software. Even within those limits, many algorithms will execute faster with smaller samples. Accurate models can often be built with as few as several thousand records. Hence, we will want to sample a subset of records for model building. Table 2.4 provides code for sampling in R.

Oversampling Rare Events in Classification Tasks

If the event we are interested in classifying is rare (e.g., customers purchasing a product in response to a mailing, or fraudulent credit card transactions), sampling a random subset of records may yield so few events (e.g., purchases) that we have little information on them. We would end up with lots of data on non-purchasers and non-fraudulent transactions but little on which to base a model that distinguishes purchasers from non-purchasers or fraudulent from non-fraudulent. In such cases, we want our sampling procedure to overweight the rare class (purchasers or frauds) relative to the majority class (non-purchasers, non-frauds) so that our sample ends up with a healthy complement of purchasers or frauds.

Assuring an adequate number of responder or "success" cases to train the model is just part of the picture. A more important factor is the **costs of misclassification**. Whenever the response rate is extremely low, we are likely to attach more importance to identifying a responder than to identifying a non-responder. In direct-response advertising (whether by traditional mail, e-mail, or web advertising), we may encounter only one or two responders for every hundred records—the value of finding such a customer far outweighs

the costs of reaching him or her. In identifying fraudulent transactions or customers unlikely to repay debt, the costs of failing to find the fraud or the nonpaying customer are likely to exceed the cost of more detailed review of a legitimate transaction or customer.

If the costs of failing to locate responders are comparable to the costs of misidentifying responders as non-responders, our models would usually achieve highest overall accuracy if they identified everyone as a non-responder. In such a case, the misclassification rate is very low—equal to the rate of responders—but the model is of no value. More generally, we want to train our model with the asymmetric costs in mind so it will catch the more valuable responders, probably at the cost of catching and misclassifying more non-responders. This subject is discussed further in Chapter 5.

Preprocessing and Cleaning the Data

Types of Variables

There are several ways of classifying variables. Variables can be:

- **Numerical** or **text** (character/string).
- **Continuous** (able to assume any real numerical value, usually in a given range), **integer** (taking only integer values), **categorical** (assuming one of a limited number of values), or **date**.

Categorical variables can be either coded as numerical (1, 2, 3) or text (e.g., "payments current," "payments not current," "bankrupt"). Categorical variables can be:

- **Nominal (unordered)**, such as *North America*, *Europe*, and *Asia*.
- **Ordinal (ordered)**, such as *high value*, *low value*, and *nil value*.

Continuous variables can be handled by most data mining routines, except for the naive Bayes classifier, which deals exclusively with categorical predictor variables. Sometimes it is desirable to convert continuous variables to categorical (e.g., for outcome variables where a numerical result is mapped to a decision threshold: *grant credit*, *start treatment*, etc.).

For the West Roxbury data, Table 2.5 presents some R statements to review the variables, determine what type (class) R thinks they are, and to determine the number of levels in a factor variable.

Handling Categorical Variables

Categorical variables can be handled by most data mining routines but often require special handling. If the categorical variable is **ordered** (age group, degree of creditworthiness, etc.), we can sometimes code the categories numerically (1, 2, 3, …) and treat the variable as if it were continuous. However, if it is a **nominal** categorical variable, it often must be decomposed into a series of binary dummy variables. For example, a single categorical variable that can have possible values **"student," "unemployed," "employed," "retired"** would be split into four separate dummy variables:

- Student—Yes/No

- Unemployed—Yes/No
- Employed—Yes/No
- Retired—Yes/No

Often, you only need three of these dummy variables in actual modeling (the "reference" or omitted category can be inferred if the other three are 0). For certain methods (e.g., linear regression and logistic regression), including all four can cause the algorithm to fail due to redundant information. Typical methods for dummy variables in R leave the original categorical variable intact, so be sure not to use both the original variable and its dummies. The R code to create binary dummies from a categorical (factor) variable is given in Table 2.6.

Variable Selection

More is not necessarily better when it comes to selecting variables for a model. **Parsimony** or **compactness** is often desirable. The more variables we include, the more complex the model, and the more records we need to estimate relationships reliably. Models with many variables can also be less robust because they require more data collection, may be prone to more data quality issues, and involve extra cleaning and preprocessing.

How Many Variables and How Much Data?

Statisticians have procedures (called **power calculations**) to learn how many records are needed to achieve a given degree of reliability in estimating an average population effect from a sample. In data mining, the goal is typically predicting individual records, which often requires larger samples. A common rule of thumb is to have around **10 records for every predictor variable**. Another rule, used by Delmaster and Hancock for classification, is:

at least $6 \times m \times p$ records where `m` is the number of outcome classes and `p` is the number of variables.

Even with ample data, it is wise to pay close attention to which variables to include. **Domain knowledge** is crucial. Suppose we're predicting total purchase amount (Y) and we discover one predictor X1 is actually "shipping cost," which is calculated as a percentage of the total purchase. Clearly, shipping cost cannot be used to predict purchase amount if it's known only after the transaction is completed. Another example: if you only have data on approved loan applicants, you cannot directly predict default at the time of loan application because you have no data on applicants you denied.

Outliers

With more data, the likelihood of encountering erroneous or extreme values increases. **Outliers** are values that lie far away from the bulk of the data. Rules of thumb, such as "anything over three standard deviations from the mean is an outlier," are common, but any real decision about whether an outlier is erroneous often requires domain knowledge. If the number of records with outliers is small, they might be treated as missing data or corrected manually if they're clearly errors (e.g., a misplaced decimal).

One way to inspect for outliers is to sort by each column and look for extreme values. Another is to examine the min and max of each column. More automated approaches (e.g., clustering) can identify records that don't "fit" with the bulk of the data.

Missing Values

If the number of records with missing values is small, dropping those records might be an option. However, even a small proportion of missing values spread across many variables can affect a large fraction of records. An alternative is **imputing** missing values (e.g., substituting the mean or median of existing values for that variable). This does not add real information about the missing record, but it keeps that record in the dataset for other variables.

If many records are missing a particular variable, either drop that variable or invest in obtaining the missing information if it's crucial. Handling missing data can be time-consuming. Sometimes a "0" might mean two different things (truly zero vs. missing), and a domain specialist is needed to clarify.

Normalizing (Standardizing) and Rescaling Data

Some algorithms require that variables be **normalized** or **rescaled**. **Normalization (standardizing)** typically subtracts the mean and divides by the standard deviation (a z-score). Another popular approach is to rescale data to a [0,1] range, subtracting the minimum and dividing by the range. This step is especially important if distance-based measures (e.g., clustering, k-nearest neighbors) are used, where variables with larger numeric scales could dominate the distance measure.

Predictive Power and Overfitting

In supervised learning, a key question is: **How well will our model perform on new data?** We want to compare the performance of various models and choose one that generalizes beyond the dataset at hand. To do this, we use **data partitioning** and try to avoid **overfitting**.

Overfitting

The more variables we include, the higher the risk of overfitting. **Overfitting** happens when a model fits not just the underlying relationship but also the random noise in the training data, leading to poor performance on new data.

A classic example is drawing a complicated curve through data points so that every point is perfectly fit, but the curve oscillates wildly and doesn't represent the real trend. When we add spurious predictor variables in a classification or regression problem (e.g., "height" predicts donations merely because the few tall people in our sample donated heavily), the model can latch onto these random coincidences.

Overfitting commonly occurs if the dataset is small relative to the number of predictors. Even if we theoretically know a higher-degree curve is correct, with too few data points a simpler function often predicts

new data better. Overfitting also arises when testing many models and picking the "best" one—some will appear better simply by chance with a particular dataset.

Creation and Use of Data Partitions

Using the same data for both fitting a model and evaluating it leads to **optimism bias**, where chance aspects of the data favor one model. The solution is to **partition** the data into separate sets and train on one while validating on the other. In a classification model, for instance, we might measure the misclassification rate on the held-back (validation) set. A model that does well on new data is said to **generalize** well.

Training Partition

The **training partition** (usually the largest portion) is used to build the various candidate models.

Validation Partition

The **validation partition** (sometimes called the test partition) is used to assess each model's performance and pick the best one. In some algorithms (e.g., classification and regression trees), the validation set may also be used repeatedly in building or tuning the model.

Test Partition

A separate **test partition** (holdout or evaluation partition) can be used for a final unbiased estimate of the chosen model's performance. After a model is chosen from among several, some "chance" aspects of the validation data may have favored that model, so measuring final performance on a new, untouched test partition provides a more realistic estimate.

Figure 2.4 illustrates the three data partitions and their use in data mining. In many cases, only two partitions (training and validation) are used, especially if finding the best model is the focus and an exact estimate of performance is less crucial.

Table 2.9 shows R code for partitioning the West Roxbury data into two or three sets. In the two-set approach, we randomly sample records into the training set, and use the remaining records as validation. In the three-set approach, we first sample training records, then sample validation from what remains, and leave the rest as the test partition.

Cross-Validation

When the dataset is small, partitioning may not be advisable because each partition will be too small. **Cross-validation** helps in this scenario. One popular method is **k-fold cross-validation**, where the data are randomly split into k non-overlapping folds (e.g., k=5, each fold is 20% of the data). The model is trained k times, each time using k−1 folds as the training set and the remaining fold as the validation set. This means every record ends up in a validation partition once. Predictions from each fold can be aggregated for an overall performance measure.

Building a Predictive Model

Let us go through the steps typical of many data mining tasks using multiple linear regression as an example. This illustrates the overall process before we tackle other algorithms.

Modeling Process

1. **Determine the Purpose**
   We want to predict the value of homes in West Roxbury for new records.
2. **Obtain the Data**
   We use the 2014 West Roxbury housing data. The dataset is small enough not to require further sampling.
3. **Explore, Clean, and Preprocess the Data**
   - Check the **data dictionary** (variable descriptions) to ensure clarity about each variable.
   - Consider **TAX**: because tax is a function of assessed value, it might not be useful if assessed value is not known.
   - Look for **outliers**. Example: a record showing 15 floors is likely an error (maybe 1.5).
   - Create **dummy variables** for categorical variables. In our example, **REMODEL** has three categories.
4. **Reduce the Data Dimension**
   The West Roxbury dataset already has a small number of variables. In larger datasets, you might condense or transform multiple variables or combine categories if appropriate.
5. **Determine the Data Mining Task**
   Our task is a **supervised prediction** of **TOTAL VALUE** using the predictor variables (excluding TAX).

6. **Partition the Data**
   We divide the data into training and validation sets as per Table 2.9. The model is built on the training set, then tested on the validation set to see how it performs on new data. (A test partition can also be used, but we omit it for simplicity here.)

7. **Choose the Technique**
   Here, we use **multiple linear regression**. After training the model on the training set, we apply it to the validation set.

8. **Use the Algorithm to Perform the Task**
   In R, we use `lm()` on the training data to predict house value, then apply the fitted model to the validation data. Table 2.11 shows predicted (fitted) values and residuals for training, and Table 2.12 for validation. Table 2.13 compares errors (e.g., mean error, root-mean-squared error, etc.). As expected, error is smaller on the training data than on the validation data.

9. **Interpret the Results**
   Typically, we would try other models (e.g., regression trees) or refine settings (e.g., best-subset variable selection) to improve error rates on the validation data. Ultimately, we pick the model with the best validation performance (and possibly the simplest form).

10. **Deploy the Model**
    Once the best model is chosen, it is used to **score** new data—predicting **TOTAL VALUE** for homes where this value is unknown.

Table 2.14 shows an example of a data frame with three new homes to be scored. All required predictor columns are present, and the output column is absent.

Using R for Data Mining on a Local Machine

A key point is that heavy-duty data mining often does not require the complete dataset of millions of records. As with polling, a sufficiently large **sample** (e.g., 20,000 records) can yield an accurate model. Therefore, each partition (training, validation, test) can typically be handled in R's available memory.

For very large datasets ("Big Data"), you might need to manage memory, removing unused objects (`rm()`) and calling garbage collection (`gc()`). Various packages in R also support parallel and large-scale computations.

Automating Data Mining Solutions

Most supervised data mining applications aim for an **ongoing** predictive or classification process, not a one-time analysis. During **prototype mode**, you explore different algorithms and fine-tune them. Once the final model is chosen, it is typically **deployed** in an automated workflow or **data pipeline**. For instance:

- The IRS processes hundreds of millions of tax returns and applies a fraud model automatically during normal processing.
- Music streaming services (e.g., Pandora, Spotify) instantly generate personalized recommendations using embedded models.

These systems require the predictive algorithm to be built into the **computational setting** via **APIs** that define how data flow in and out. After deployment, models must be monitored and periodically **re-evaluated** because conditions change over time (e.g., consumer behavior, economic factors), and predictive performance can degrade. Models often have short shelf lives—sometimes only a year. When performance declines, we return to prototype mode and create a fresh model.

Although our focus here is on the **prototyping phase**—defining, developing, and selecting models—be aware that the real effort in practice often lies in automated deployment and its maintenance.

House management

More to Read

Assignment

- What to do
- Requirement
  - PDF format
  - file name should be include your student id and name
    - stuID_name_title.pdf (e.g. 1111111_ChungilChae_SelfIntroduction.pdf)
- Due date
  - by DATE 11:59PM
  - NO LATE SUBMISSION ALLOWED!!!!

Reference

Shmueli, G., Bruce, P. C., Yahav, I., Patel, N. R., & Lichtendahl Jr, K. C. (2017). *Data mining for business analytics: Concepts, techniques, and applications in r*. John Wiley & Sons.

